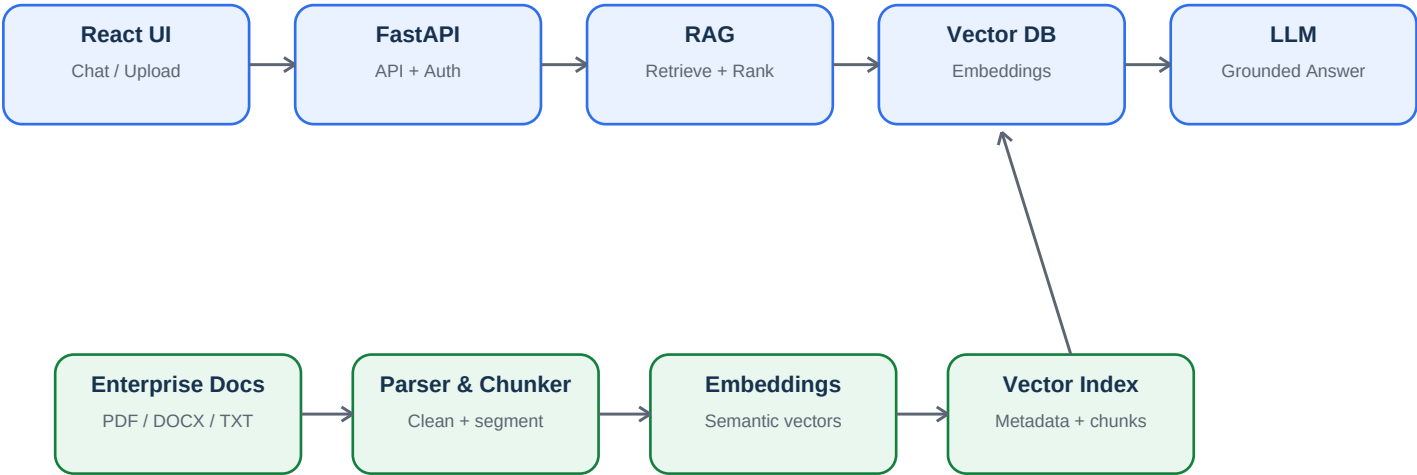


ENTERPRISE GENAI KNOWLEDGE ASSISTANT

Production-oriented Retrieval-Augmented Generation (RAG) solution for enterprise knowledge discovery

Portfolio Project 01	AI Solutions Architect
Generative AI • LLM • RAG • Semantic Search	Python • FastAPI • React • Vector Database • Docker • Cloud



High-level reference architecture — implementation choices can be adapted to the client's cloud and data estate.

Portfolio note: This document presents a portfolio/reference implementation and architecture demonstration. It is intentionally written so it can be adapted to a client's documents, APIs, security model, LLM provider, and cloud environment. Only measured results from an actual implementation should be presented as production metrics.

1. Executive Summary

The Enterprise GenAI Knowledge Assistant enables users to ask natural-language questions against an organization's private knowledge base. Instead of relying on an LLM's general training data alone, the solution retrieves relevant enterprise content and supplies that evidence to the model before response generation.

The architecture separates the user experience, API layer, retrieval pipeline, vector index, and LLM integration. This makes the solution suitable as a foundation for internal knowledge assistants, customer-support copilots, technical documentation assistants, operations assistants, and enterprise search modernization.

01	02	03	04
Document Ingestion	Semantic Retrieval	Grounded LLM Answers	API + Cloud Deployment

Business value

- **Faster knowledge discovery:** users can ask questions in natural language rather than manually searching multiple documents.
- **Grounded responses:** retrieval provides relevant enterprise evidence to reduce unsupported or generic answers.
- **Reusable architecture:** the same pattern can support policies, product documentation, operational procedures, technical manuals, and support knowledge.
- **Enterprise-ready foundation:** the design includes API integration, access control, observability, containerization, and scalable deployment considerations.

Representative use cases

Use case	Example question	Value
Internal knowledge assistant	What is the approved deployment process?	Reduce time spent searching documentation
Technical support copilot	Why did this service deployment fail?	Accelerate troubleshooting
Operations assistant	What is the procedure for a regional outage?	Surface approved operational guidance
Product documentation assistant	How do I configure feature X?	Improve self-service support

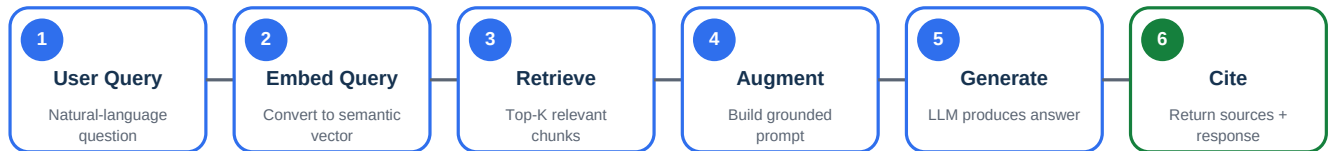
2. Problem Statement & Solution

The enterprise knowledge problem

Enterprise knowledge is often distributed across PDFs, DOCX files, wikis, operational guides, API references, and internal documentation. Keyword search can be brittle when users do not know the exact terminology used in the source material. A generic chatbot has the opposite problem: it can produce fluent answers without being grounded in the organization's private information.

Solution approach

The assistant uses a Retrieval-Augmented Generation pattern. Documents are processed into searchable chunks, each chunk is represented as an embedding and stored in a vector index. At question time, the query is embedded and matched against the knowledge base. The highest-relevance evidence is then supplied to the LLM as context.

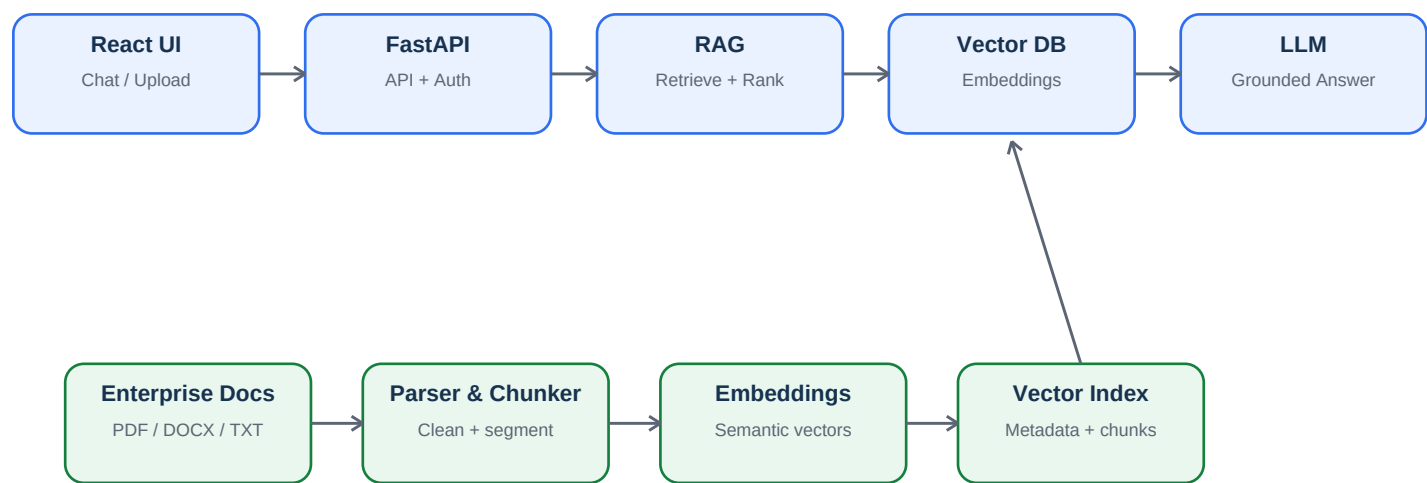


Grounding pattern: retrieve relevant evidence first, then generate an answer constrained by that evidence.

Core design principles

Principle	Implementation intent
Grounded generation	Retrieve enterprise evidence before generating a response.
Separation of concerns	Keep UI, API, retrieval, data, and model integration independently replaceable.
Security by design	Enforce authentication, authorization, tenant/data boundaries, and secret management.
Observability	Capture retrieval quality, latency, errors, model usage, and operational events.
Cloud portability	Containerize services and keep provider-specific integrations behind clear interfaces.

3. Solution Architecture



High-level reference architecture — implementation choices can be adapted to the client's cloud and data estate.

Component responsibilities

Layer	Responsibility	Representative technologies
React UI	Chat experience, document upload, source display, conversation history	React / TypeScript
FastAPI	Authentication boundary, REST endpoints, orchestration, request validation	Python / FastAPI
RAG service	Query embedding, retrieval, ranking, prompt assembly	Python / RAG framework
Vector database	Semantic index for document chunks and metadata	PostgreSQL + pgvector / managed vector DB
LLM gateway	Model abstraction, prompt policy, token/cost controls	OpenAI / Azure OpenAI / other provider
Document pipeline	Parsing, chunking, metadata, embeddings	Python document processing
Cloud runtime	Container execution, networking, scaling, monitoring	Docker / Kubernetes / Azure / AWS

Why this architecture is extensible

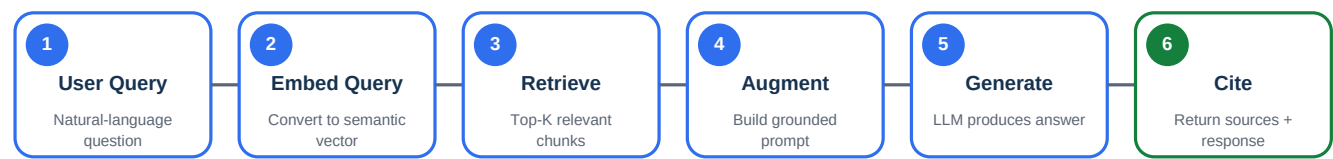
The LLM provider, vector store, document parser, and cloud runtime can be replaced without redesigning the entire application. The same API and retrieval contracts can support different models or enterprise data sources as requirements evolve.

4. RAG Pipeline & Knowledge Processing

Offline / ingestion pipeline

- **Ingest:** accept approved enterprise documents and associated metadata.
- **Parse:** extract text while preserving useful structure such as headings, pages, sections, and document identifiers.
- **Chunk:** split content into retrieval-friendly segments with controlled overlap.
- **Embed:** generate semantic vectors for each chunk.
- **Index:** store vectors, text, metadata, permissions, and source references.
- **Version:** track document versions so stale content can be removed or superseded.

Online question-answering pipeline



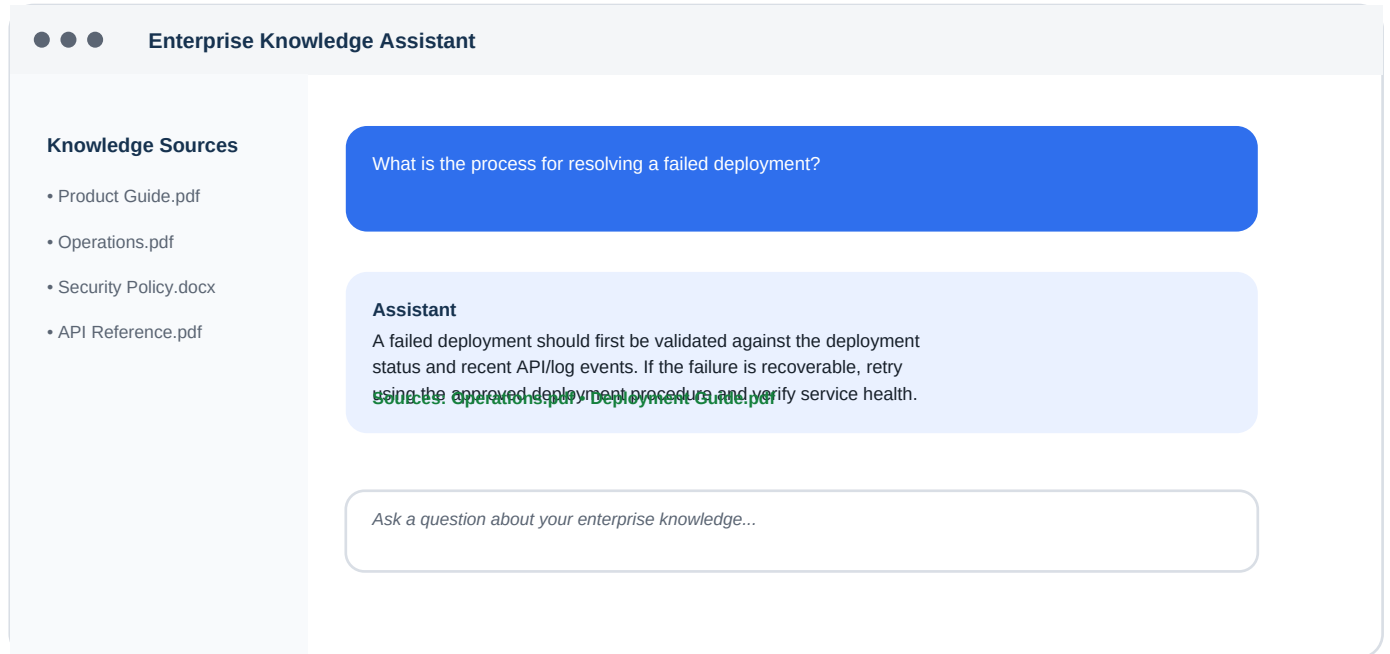
Grounding pattern: retrieve relevant evidence first, then generate an answer constrained by that evidence.

Retrieval quality controls

Control	Purpose
Top-K retrieval	Limit context to the most relevant evidence.
Metadata filtering	Restrict retrieval by tenant, department, document type, version, or access policy.
Re-ranking	Improve ordering when vector similarity alone is insufficient.
Similarity threshold	Avoid confidently answering when evidence is too weak.
Source attribution	Return document/source references alongside the answer.
No-answer policy	Prefer 'insufficient evidence' over unsupported generation.

5. User Experience & API Design

Representative user experience



UX objective: make the assistant useful and trustworthy by showing not only the generated response, but also the evidence used to produce it.

Representative REST API contract

```
POST /api/v1/knowledge/query
{
  "question": "What is the approved deployment process?",
  "filters": {
    "document_type": ["operations", "deployment"],
    "version": "current"
  },
  "top_k": 5
}
```

```
Response
{
  "answer": "...grounded response...",
  "sources": [
    {
      "document": "Deployment Guide.pdf",
      "section": "Release Procedure",
      "relevance": 0.91
    }
  ],
  "request_id": "..."
}
```

Representative document-ingestion endpoint

```
POST /api/v1/knowledge/documents
Content-Type: multipart/form-data

Fields:
- file
- document_type
- department
```

- version
- access_policy

Pipeline:

upload → parse → chunk → embed → index → validate

6. Enterprise Security, Reliability & Governance

Security controls

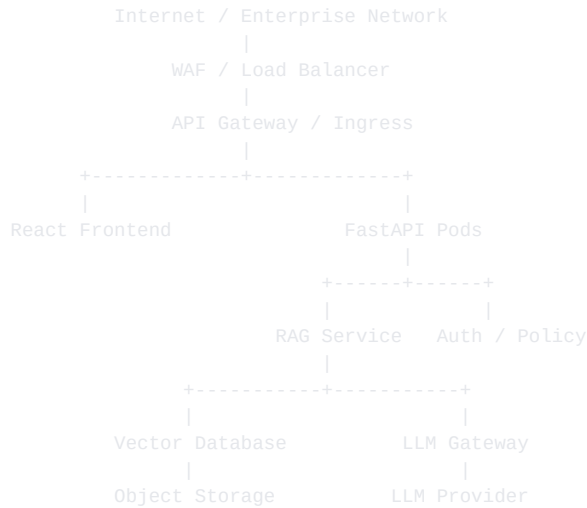
Area	Design approach
Authentication	OIDC/OAuth2 or enterprise identity provider at the API boundary.
Authorization	Role- and attribute-based access controls; retrieval filtered by user permissions.
Data isolation	Tenant / department metadata enforced before retrieval.
Secrets	Cloud secret manager or Kubernetes secrets; no credentials in source code.
Encryption	TLS in transit and encryption at rest for indexed content and application data.
PII protection	Optional redaction/classification before content reaches the model provider.
Prompt injection	Treat retrieved content as untrusted data; separate instructions from evidence and validate tool access.
Auditability	Log user/request IDs, document IDs, retrieval events, model usage, and administrative actions.

Reliability & failure handling

- Timeouts and bounded retries for model and vector-store calls.
- Circuit-breaker / fallback behavior when downstream AI services are unavailable.
- Graceful 'insufficient evidence' responses when retrieval confidence is low.
- Idempotent ingestion operations to avoid duplicate document indexing.
- Request correlation IDs for end-to-end troubleshooting.
- Health checks, readiness checks, structured logs, metrics, and alerting.

7. Cloud Deployment & Observability

Containerized deployment model



Deployment options:

- Docker for local / POC execution
- Kubernetes for scalable production deployment
- Azure / AWS managed services as appropriate

Observability model

Signal	Examples
Latency	API latency, retrieval latency, model latency, end-to-end response time
Quality	Retrieval hit rate, source relevance, no-answer rate, user feedback
Reliability	Error rate, timeout rate, retry count, dependency availability
AI usage	Token consumption, model selection, request volume, estimated cost
Security	Authentication failures, denied retrievals, suspicious prompts, policy violations

Scalability considerations

- Keep API and retrieval services stateless so they can scale horizontally.
- Cache repeated embeddings and frequently requested knowledge where appropriate.
- Use asynchronous ingestion for large document batches.
- Partition vector indexes or metadata by tenant/domain when data volume or isolation requires it.
- Use model routing and bounded context to manage LLM latency and cost.

8. Representative Implementation

FastAPI orchestration pattern

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Query(BaseModel):
    question: str
    top_k: int = 5

@app.post("/api/v1/knowledge/query")
def query_knowledge(req: Query):
    query_vector = embed(req.question)
    chunks = vector_search(query_vector, top_k=req.top_k)
    context = build_context(chunks)
    answer = generate_grounded_answer(req.question, context)
    return {
        "answer": answer,
        "sources": build_sources(chunks)
    }
```

RAG orchestration pattern

```
def generate_grounded_answer(question, chunks):
    if not chunks or below_similarity_threshold(chunks):
        return "I could not find sufficient evidence in the knowledge base."

    prompt = build_prompt(
        system_policy=GROUNDING_POLICY,
        question=question,
        evidence=chunks
    )

    return llm.generate(
        prompt=prompt,
        temperature=0.1
    )
```

Engineering considerations

Concern	Recommended approach
Prompt control	Version prompts, separate system policy from retrieved content, test adversarial inputs.
Evaluation	Maintain a curated question/evidence set and measure retrieval and answer quality.
Versioning	Version embeddings, prompts, parsers, indexes, and document sources.
Testing	Unit test parsing/retrieval; integration test API + vector DB + model boundary; load test end-to-end paths.
Provider abstraction	Use an LLM gateway/interface so model providers can be changed without rewriting business logic.

9. Delivery Plan, Testing & Acceptance Criteria

Suggested delivery phases

Phase	Scope	Output
1. Discovery	Use cases, data sources, access model, target users	Requirements + architecture
2. Knowledge pipeline	Parsing, chunking, embeddings, vector indexing	Ingestion pipeline
3. RAG API	Retrieval, prompt orchestration, source attribution	Working AI API
4. User experience	Chat UI, upload, source display, error states	React application
5. Security & operations	Identity, authorization, logging, monitoring	Enterprise controls
6. Deployment	Containerization, CI/CD, cloud runtime	Deployable environment
7. Evaluation	Golden questions, retrieval tests, latency and reliability tests	Evaluation report + roadmap

Acceptance criteria

- ✓ Users can upload approved document types and see ingestion status.
- ✓ Users can ask natural-language questions and receive relevant answers.
- ✓ Responses expose the source documents/sections used for grounding.
- ✓ Unauthorized users cannot retrieve restricted document content.
- ✓ The API returns predictable error responses for invalid requests and downstream failures.
- ✓ Application logs and request IDs support troubleshooting.
- ✓ The solution can be containerized and deployed consistently across environments.
- ✓ Evaluation includes both retrieval quality and answer quality rather than relying only on subjective review.

Evaluation framework

Metric	What it measures	How to validate
Retrieval relevance	Whether retrieved chunks contain the evidence needed.	Golden dataset + human/automated relevance scoring.
Groundedness	Whether the answer is supported by retrieved evidence.	Evidence-based evaluation against reference answers.
Latency	User-perceived response speed.	Load and integration tests; track p50/p95/p99.
Reliability	Behavior under failures and dependency issues.	Fault injection + retry/timeout testing.

10. Portfolio Summary

This project demonstrates the ability to design an enterprise-oriented Generative AI solution rather than a simple chatbot. The implementation pattern covers the complete path from private knowledge ingestion to semantic retrieval, grounded LLM generation, source attribution, API integration, security, observability, and cloud deployment.

Capabilities demonstrated

- Generative AI and LLM solution architecture
- Retrieval-Augmented Generation (RAG)
- Semantic search and vector retrieval
- Python and FastAPI service development
- React-based AI user experience
- REST API design and integration
- Document ingestion and knowledge processing
- Enterprise security and access control
- Docker and cloud-native deployment
- Scalability, reliability, and observability
- AI evaluation and production-readiness considerations

Suggested client engagement

Discovery → Architecture → POC → Production. A typical engagement can begin with a focused knowledge-assistant use case, validate retrieval and answer quality using a representative document set, then extend the architecture with enterprise identity, access control, monitoring, integrations, and production deployment.

Recommended portfolio attachments

Asset	Purpose
This architecture PDF	Shows solution design and engineering depth.
Architecture diagram PNG	Provides a quick visual summary for clients.
60–90 second demo video	Demonstrates the working user flow.
GitHub repository (if public)	Provides implementation evidence and code quality.
Live demo (if available)	Lets clients experience the solution directly.

Positioning: Enterprise AI Solutions Architecture • GenAI • LLM • RAG • AI Integration • Python • Cloud • Enterprise Modernization

For an actual client project, implementation details, cloud services, model provider, security controls, and performance figures should be customized to the client's environment and validated through testing.